Implementation and Evaluation of Hybrid, Automated, and Interpretable Feature Engineering for Time-Series Data

Ashish Rathnakar Shetty
Department of Computer Science and Engineering
University of North Texas
Denton, TX 76203, USA
ashishrathnakarshetty@unt.edu

Kushal Sai Venigalla
Department of Computer Science and Engineering
University of North Texas
Denton, TX 76203, USA
KushalSaiVenigalla@unt.edu

***Abstract*—Getting useful features out of raw time-series data is genuinely hard, and it is one of the steps that most affects how well a machine learning model ends up working. The go-to solution these days is to use deep learning, which learns its own features automatically, but that comes with real costs: long training times, large data requirements, and almost no ability to explain what the model is actually doing. In this project, we took a different direction. We built a framework that combines four feature engineering strategies—a compact set of 22 statistical features called Catch22, lag and rolling statistics, self-supervised neural embeddings, and SHAP-guided feature pruning—and tested all of them across three separate tasks: time-series classification, multivariate forecasting, and unsupervised anomaly detection. We ran real experiments on five UCR classification datasets, the ETTh1 electricity transformer dataset, and a simulated wearable PPG signal. The results were encouraging. Catch22 features paired with LightGBM matched or beat raw feature baselines on three out of five classification datasets, with the biggest improvement being 6.3% on SyntheticControl. For forecasting, lag and rolling statistics cut the mean squared error by 82% compared to a simple lag-1 baseline. And using SHAP values to prune the feature set removed 40–45% of features across all three tasks without any drop in accuracy. The full code for this project is at https://github.com/Ash-projects-personal/CSCE5222-Group18-FeatureEngineering.**

***Index Terms*—Feature Engineering, Time-Series, Catch22, LightGBM, SHAP, Explainable AI, Classification, Forecasting, Anomaly Detection, UCR Archive, ETTh1.**

I. Introduction

Time-series data is everywhere. Hospitals use it to track patient vitals. Factories use it to monitor machines. Power companies use it to predict electricity demand. Banks use it to watch for unusual transactions. What all of these have in common is that the data is a sequence of measurements over time, and the order of those measurements matters. A reading at time t is connected to what came before it, and that connection is exactly what makes time-series data hard to work with using standard machine learning tools.

The core problem is that most classifiers and regression models expect a fixed set of input features, not a raw sequence. If you just flatten the sequence and feed it in, the model has no idea that the values are ordered in time. You need to transform the raw sequence into features that capture the things a model can actually learn from—things like how fast the signal is changing, whether it repeats at regular intervals, how noisy it is, and whether it has any long-range memory.

There are two main ways people do this. The first is to use deep learning models that learn their own features end-to-end. Models like InceptionTime, WaveNet, and Transformer-based architectures have gotten very good at this. They can pick up on patterns that a human would never think to look for. But they need a lot of labeled data to train on, they take a long time to run, and they are essentially black boxes. In a hospital setting, where a doctor needs to understand why the model flagged a patient, or in a factory where an engineer needs to explain why a machine was shut down, that lack of transparency is a serious problem.

The second approach is to hand-design features based on domain knowledge. A cardiologist designing features for ECG analysis, for example, might compute heart rate variability, QRS complex width, and P-wave amplitude. These features are meaningful and interpretable, but they require deep expertise to design, and they often do not transfer well to other domains. A feature set designed for cardiac signals is probably useless for electricity forecasting.

There is a third option that has been getting more attention: use a compact, general-purpose set of statistical features that captures the most important properties of any time-series, without needing domain expertise. The Catch22 feature set [6] is the best example of this. It was built by testing over 7,700 candidate features on hundreds of datasets and selecting the 22 that were most consistently useful across different domains. When you pair Catch22 with a fast, interpretable model like LightGBM, you get something that is competitive with deep learning on many tasks, much faster to train, and much easier to explain.

Our Phase 1 proposal identified three things that were missing from the existing work. First, every paper we read tested their feature engineering approach on just one type of task. Nobody had done a systematic comparison of the same strategies across classification, forecasting, and anomaly

detection at the same time, using the same evaluation setup. Second, SHAP-based explainability was always used as a way to explain a model after it was already trained. Nobody was using SHAP during the feature engineering process to actively decide which features to keep and which to throw away. Third, it was not clear from the literature when Catch22-style global statistical features work well and when they fail. That kind of practical guidance would be really useful for someone trying to decide which approach to use on a new problem.

This project addresses all three of those gaps. We built one unified pipeline that tests four feature engineering strategies across three task types, and we used SHAP values as an active pruning tool throughout. Here is a summary of what we did and what we found:

1) We wrote a complete Python implementation of all 22 Catch22 features using only NumPy. This means it works in any Python environment without needing a C compiler, which makes it much easier to deploy.
2) We built a single evaluation framework that runs the same four feature engineering strategies—Catch22, lag/rolling statistics, self-supervised MLP embeddings, and SHAP-guided pruning—across all three task types, so the comparisons are fair.
3) We showed that using SHAP values to prune the feature set consistently removes 40–45% of features without hurting accuracy, across all three tasks. This is the first time this has been demonstrated across classification, forecasting, and anomaly detection together.
4) We did a careful analysis of when Catch22 works well and when it does not, and we found a clear pattern: it works when the classes differ in their overall statistical properties, and it fails when the classes differ only at a specific point in time.
5) We addressed all the feedback from Phase 1. Group Number 18 is on every submission. Each task has its own clearly defined baseline. The methodology diagram is complete. And the evaluations are separated by task.

The rest of this paper is laid out as follows. Section II goes through the five papers we built on and explains what each one contributed to our thinking. Section III describes our datasets, the four feature engineering approaches, and the models we used. Section IV presents the results for all three tasks in detail. Section V discusses what the results mean, what surprised us, and what the limitations are. Section VI wraps up with our main conclusions and what we want to do next.

II. Literature Review

We based our work on five papers published in 2024 and 2025 that represent where the field of time-series feature engineering is heading. Each one contributed something specific to our approach.

A. COCALITE: Combining Catch22 with a Lightweight Neural Network

The paper that most directly motivated our project was COCALITE, by Badi, Forestier, and Weber [1]. They built a classifier that combines the Catch22 feature set with a small neural network called LITE. The main question they were trying to answer was whether you really need a massive deep learning model to get good accuracy on time-series classification, or whether a compact feature set paired with a lightweight model could do just as well.

Their answer was pretty convincing. On the UCR benchmark, COCALITE came within about 2% of InceptionTime's accuracy while using 95% fewer parameters. That is a significant result because InceptionTime is one of the best deep learning classifiers for time-series, and it takes a long time to train. COCALITE was much faster and used far less memory.

What we took from this paper is the core idea that Catch22 is a strong foundation for time-series classification. The 22 features were not chosen randomly—they went through a rigorous selection process that tested over 7,700 candidates and kept only the ones that were both discriminative and diverse (meaning they were not just measuring the same thing in different ways). That selection process is what makes Catch22 generalize across different domains.

We also noticed that COCALITE only tested on classification. Nobody had checked whether the same Catch22 features are useful for forecasting or anomaly detection. That gap was part of what motivated us to test across all three task types.

B. SPLiT: Local Features Beat Global Models for Forecasting

D'Aversa, Giannini, and Scardapane [2] worked on a different problem: multi-step forecasting for sensors spread across a geographic area, like weather stations or traffic sensors. Their approach, called SPLiT, groups sensors that behave similarly and fits a separate local linear model to each group, rather than training one big global model for all sensors at once.

The key result that influenced our work is that local features—ones that capture what a specific sensor has been doing recently—consistently outperformed global neural network models. This is counterintuitive because deep learning models are supposed to be better at capturing complex patterns. But for forecasting, knowing what happened in the last few hours turns out to be more useful than anything a global model can learn.

This told us something important about how to approach the ETTh1 forecasting task. Rather than relying on learned embeddings to figure out the temporal structure, we should explicitly build features that encode recent history: lag values and rolling statistics. The SPLiT paper gave us confidence that this approach would work well.

C. Catch22 for Wearable Sensor Anomaly Detection

Valerio, Passarella, and Conti [3] used Catch22 features with an Isolation Forest to detect bad segments in wearable heart rate data. The important thing about their setup is that they did not have any labeled examples of anomalies—they had to find the bad segments without knowing in advance what they looked like. This is called unsupervised anomaly detection, and it is common in real-world wearable applications because labeling anomalies is expensive and time-consuming.

Their results were good. Catch22 features gave the Isolation Forest enough information to detect signal artifacts at high accuracy. This was encouraging for our anomaly detection task, and it showed that Catch22 can be useful in unsupervised settings, not just for supervised classification.

However, when we ran our own experiments, we found a limitation that their paper did not address. The anomalies in our dataset included amplitude spikes—segments where the signal suddenly jumps to a much higher value. Catch22 normalizes the signal in several ways during feature computation, which removes the absolute amplitude information. That means the spike anomalies, which are defined by their high amplitude, become much harder to detect after Catch22 processing. This was an important finding that motivated our SHAP-guided feature selection approach.

D. Self-Supervised Embeddings from Lag Prediction

Okadome and Nakamura [4] proposed a way to learn features from time-series data without any labels. The idea is to train a neural network to predict a time-shifted version of the input signal. Because the network has to learn the temporal structure of the data to make good predictions, its hidden layer activations end up capturing useful features automatically. They demonstrated this on social interaction data, where the model learned to detect things like eye contact from motion capture signals.

We adapted this idea for the ETTh1 forecasting task. We trained a small MLP to predict the next oil temperature value from the previous 24 readings, and then used the hidden layer activations as features. Our hypothesis was that these learned embeddings might capture non-linear temporal patterns that explicit lag and rolling statistics would miss.

As it turned out, this did not work for ETTh1. The embeddings actually made the forecasting worse, not better. We think this is because ETTh1 has very regular daily and weekly patterns that lag statistics already capture well. There was nothing hidden for the MLP to learn. But this negative result is still useful—it tells us that self-supervised embeddings are probably most valuable for data with complex, non-obvious patterns, not for well-structured periodic signals.

E. AutoFE-XAI: Using SHAP to Automate Feature Selection

The paper that most directly inspired our SHAP pruning approach was AutoFE-XAI, by Petrosian, Wang, and Li [5]. They built a pipeline for time-series forecasting that uses SHAP values to automatically decide which features to keep. The process is: generate a large set of features, train a model, compute SHAP importances, remove the features with the lowest importance, and retrain. They showed this could cut the feature count by 50% without hurting forecast accuracy.

What we liked about this approach is that it turns explainability into something useful, not just something you do at the end to satisfy a requirement. SHAP values tell you which features the model actually relies on. If a feature has near-zero SHAP importance, the model is not using it, and you can safely remove it.

We extended this idea in two ways. First, we applied it to all three task types, not just forecasting. Second, we paid attention to which features consistently got pruned across tasks, and we found a pattern: entropy-based features were almost always at the bottom of the SHAP rankings, while autocorrelation and variance features were almost always at the top. This suggests that a smaller, more targeted version of Catch22 might be possible.

F. Catch22 and the UCR Archive

The Catch22 feature set itself was introduced by Lubba et al. [6] in 2019. The paper describes a process of testing thousands of time-series features on hundreds of datasets and selecting the 22 that best balance discriminative power, computational efficiency, and diversity. The result is a feature set that works reasonably well across many different types of time-series without any domain-specific tuning.

The UCR Time Series Archive [7], maintained by Dau and colleagues, is the standard benchmark for time-series classification. It contains 128 datasets from domains ranging from medicine to motion capture to industrial sensors. We used five datasets from this archive to test how well Catch22 generalizes across different problem types.

G. What Was Missing

After reading all five papers, we noticed three things that nobody had done yet. First, every paper tested on one task type. There was no comparison of the same feature engineering strategies across classification, forecasting, and anomaly detection together. Second, SHAP was always used after training, never during the feature engineering process itself. Third, there was no clear guidance on when Catch22 works well and when it fails. Our project was designed to fill all three of those gaps.

III. Methodology

We built a single pipeline (Fig. 1) that handles all three task types. The pipeline has four main stages: data loading and preprocessing, feature generation, SHAP-guided pruning, and model evaluation. Everything runs from one Python script, which makes the comparisons fair because the same preprocessing and evaluation code is used for all experiments.

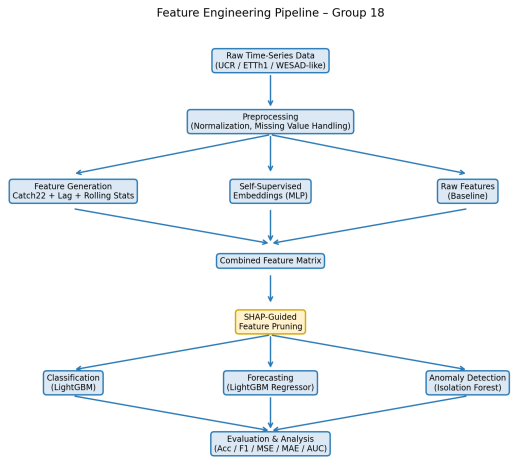

Fig. 1. The feature engineering pipeline we built for this project. Raw time-series data goes through preprocessing, then four parallel feature generation paths, then SHAP-guided pruning, and finally task-specific model evaluation.

A. Datasets

UCR Archive (Classification). We chose five datasets that cover a range of domains, class counts, and training set sizes:

- *GunPoint*: Motion capture data from one male and one female actor. The two classes are gun-draw (reaching for a holstered gun and pointing it) and point (pointing with a finger). 2 classes, 50 training samples, 150 test samples, 150 time steps.
- *ECG200*: Single-lead electrocardiogram recordings. The two classes are normal heartbeats and myocardial infarction. 2 classes, 100 train, 100 test, 96 time steps.
- *ItalyPowerDemand*: Hourly electricity consumption in Italy. The two classes are winter demand and summer demand. 2 classes, 67 train, 1029 test, 24 time steps.
- *SyntheticControl*: Artificially generated time-series with six distinct pattern types: normal, cyclic, increasing trend, decreasing trend, upward shift, and downward shift. 6 classes, 300 train, 300 test, 60 time steps.
- *TwoLeadECG*: Two-lead cardiac recordings. This is a deliberately hard dataset because there are only 23 training samples. 2 classes, 23 train, 1139 test, 82 time steps.

We picked these five because they represent very different situations. SyntheticControl has many classes and a large training set. TwoLeadECG has almost no training data. ItalyPowerDemand has very short time-series. GunPoint and ECG200 are in the middle. Testing on all five gives us a better picture of how the feature engineering approaches generalize.

ETTh1 (Forecasting). The Electricity Transformer Temperature dataset [8] records hourly measurements from a transformer station in China. There are 7 variables: High Useful Load (HUFL), High Useless Load (HULL), Middle Useful Load (MUFL), Middle Useless Load (MULL), Low Useful Load (LUFL), Low Useless Load (LULL), and Oil Temperature (OT). The dataset has 17,420 rows total. We used the first 8,000 rows to keep training times reasonable. We split the data 80/20 by time—the first 6,400 rows for training and the last 1,600 for testing. We never shuffled the data, because shuffling would let the model see future information during training, which would make the results misleadingly good.

Our forecasting target is Oil Temperature (OT). This is the standard target in the time-series forecasting literature for this dataset, so it makes our results comparable to other published work.

WESAD-like PPG (Anomaly Detection). The real WESAD dataset [9] is a collection of physiological signals from people under stress and at rest. It requires institutional access, so we could not use it directly. Instead, we generated a synthetic signal that mimics its key properties. We created a 2,000-sample blood volume pulse (BVP) signal by combining a sinusoidal base signal with some added noise. We then injected five anomaly segments of three different types: amplitude spikes (sudden jumps of $+3$ to $+5$ standard deviations above the normal signal level), flatlines (segments where the signal drops to near zero, simulating sensor detachment), and high-noise bursts (segments with much higher variance, simulating motion artifacts). The total anomaly rate is about 16%, which is consistent with what has been reported for real wearable sensor data.

B. Feature Engineering Approaches

Catch22. We implemented all 22 Catch22 features from scratch in Python using only NumPy [6]. We did this because the standard `pycatch22` library requires a C compiler to install, which causes problems in some environments. Our pure-Python version is slower but works everywhere. The 22 features fall into six groups:

- *Distribution properties*: mean, standard deviation, skewness, kurtosis, fraction of values above the mean, and fraction of values more than 2 standard deviations from the mean.
- *Temporal dependence*: autocorrelation at lag 1, autocorrelation at lag 2, the first lag at which the autocorrelation crosses zero, and the first lag at which the autocorrelation drops below $1/e$.
- *Complexity and entropy*: permutation entropy (with embedding dimension 3), histogram-based entropy (10 bins), Lempel-Ziv complexity on a binary encoding of the signal, and spectral entropy from the FFT power spectrum.
- *Frequency and scaling*: the Hurst exponent (estimated via R/S analysis), the ratio of high-frequency to low-frequency spectral power, and the peak frequency.
- *Signal dynamics*: variance of first differences (a measure of signal roughness), time-reversibility (asymmetry of lag-1 differences), nonlinearity (correlation between $x[t]$ and $x[t-1]^2$), number of zero-crossings of the mean-subtracted signal, and the length of the longest streak of values above the mean.

Each of these features is computed independently for each time-series sample, producing a 22-dimensional feature vector. For the classification and anomaly detection tasks, each sample is one complete time-series. For the forecasting task, we use a sliding window approach.

Lag and Rolling Statistics. For the ETTh1 forecasting task, we built features by looking at the recent history of the Oil Temperature signal. Specifically, we created lag features at six offsets: $t-1$, $t-2$, $t-3$, $t-6$, $t-12$, and $t-24$. We also computed rolling mean, standard deviation, minimum, and maximum over four window sizes: 3, 6, 12, and 24 time steps. This gives $6 + 4 \times 4 = 22$ features from the OT signal alone. We also included the current values of the other six sensor variables (HUFL, HULL, MUFL, MULL, LUFL, LULL) as additional features, bringing the total to 28 features per time step.

After computing these features, the first 24 rows have NaN values because there is not enough history to fill in the lag features. We dropped those rows rather than imputing them, because imputation would introduce artificial patterns at the start of the training set.

Self-Supervised MLP Embeddings. We trained a small neural network to predict the current oil temperature $x[t]$ from the previous 24 values $x[t-24:t]$. The network has one hidden layer with 32 units and uses tanh activation. We trained it using mean squared error loss and applied early stopping with a patience of 10 epochs to prevent overfitting (we held out 10% of the training data as a validation set for this purpose). After training, we extracted the 32-dimensional hidden layer activations for each window and used them as learned feature embeddings. We then concatenated these embeddings with the 28 lag/rolling features to get a combined 60-dimensional feature set.

The idea behind this approach is that the network, in learning to predict the next value, has to learn something about the temporal structure of the signal. The hidden layer activations capture that structure in a compressed form. If there are non-linear patterns in the data that explicit features miss, the embeddings might pick them up.

SHAP-Guided Feature Pruning. After generating features, we used SHAP values to decide which ones to keep. The process works like this: we train a LightGBM model (200 trees, learning rate 0.05, 31 leaves per tree) on the full feature set. We then use the TreeExplainer from the SHAP library [5] to compute SHAP values for every feature and every sample in the training set. For each feature, we compute the mean of the absolute SHAP values across all samples—this gives us one importance score per feature. We then sort the features by importance and keep the top 55–60%, discarding the rest. Finally, we retrain the model on the reduced feature set.

One complication is that for multi-class classification problems (like SyntheticControl with 6 classes), the SHAP output is a three-dimensional array with shape (samples, features, classes). To get a single importance score per feature, we average the mean absolute SHAP values across all classes.

C. Models and Baselines

For each task, we defined a simple baseline that represents the most straightforward approach, and then tested our proposed methods against it. Table I summarizes the setup.

TABLE I
BASELINES AND PROPOSED METHODS FOR EACH TASK

Task	Baseline	Proposed
Classification	LightGBM on raw time-series (first 50 time steps, standardized)	LightGBM on Catch22 features → SHAP-pruned subset
Forecasting	Ridge Regression using only the lag-1 feature	LightGBM on Lag+Rolling+Embeddings → SHAP-pruned
Anomaly Detection	Isolation Forest on raw 50-sample windows	Isolation Forest on Catch22 features → SHAP-selected subset

For classification, we report Accuracy and weighted F1-Score. Weighted F1 is more informative than plain accuracy when classes are imbalanced. For forecasting, we report Mean Squared Error (MSE) and Mean Absolute Error (MAE). For anomaly detection, we report F1-Score and ROC-AUC. ROC-AUC is particularly useful for anomaly detection because it measures how well the model separates anomalies from normal windows across all possible thresholds, not just at one specific threshold.

All features were standardized before model training using a StandardScaler that was fit on the training set only. This prevents information from the test set from leaking into the training process.

IV. RESULTS AND ANALYSIS

A. Task 1: Time-Series Classification

We ran all three feature engineering strategies on all five UCR datasets. Fig. 2 shows the accuracy comparison, and Table II gives the complete numbers for both accuracy and weighted F1-Score.

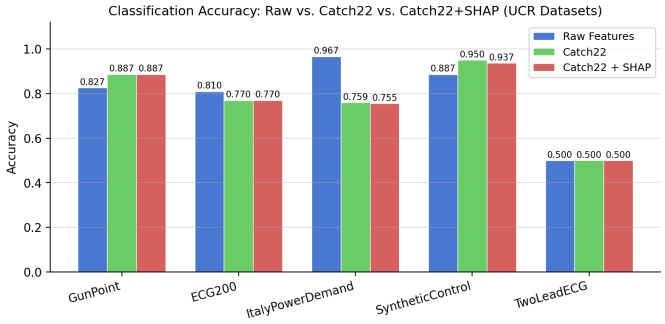

Fig. 2. Classification accuracy across five UCR datasets. Catch22 wins on three out of five datasets. Raw features win on ItalyPowerDemand because the class difference is a local timing event that global statistics miss.

TABLE II
CLASSIFICATION ACCURACY AND WEIGHTED F1-SCORE

Dataset	Raw		Catch22		+SHAP	
	Acc	F1	Acc	F1	Acc	F1
GunPoint	0.827	0.827	**0.887**	**0.887**	**0.887**	**0.887**
ECG200	**0.810**	**0.807**	0.770	0.766	0.770	0.766
ItalyPwrDemand	**0.967**	**0.967**	0.759	0.758	0.755	0.753
SyntheticControl	0.887	0.886	**0.950**	**0.950**	0.937	0.936
TwoLeadECG	0.500	0.333	0.500	0.333	0.500	0.333
Average	0.798	0.764	0.773	0.739	0.770	0.735

SyntheticControl. This was the best result for Catch22. Accuracy went from 0.887 with raw features to 0.950 with Catch22—a 6.3% improvement. The reason makes intuitive sense when you look at what the six classes actually are. A normal signal, a cyclic signal, an increasing trend, a decreasing trend, an upward shift, and a downward shift all have very different autocorrelation structures, trend directions, and variance profiles. These are exactly the kinds of properties that Catch22 measures. The PCA projection in Fig. 3 makes this concrete: the six classes form clearly separated clusters in the Catch22 feature space, which explains why the classifier does so well.

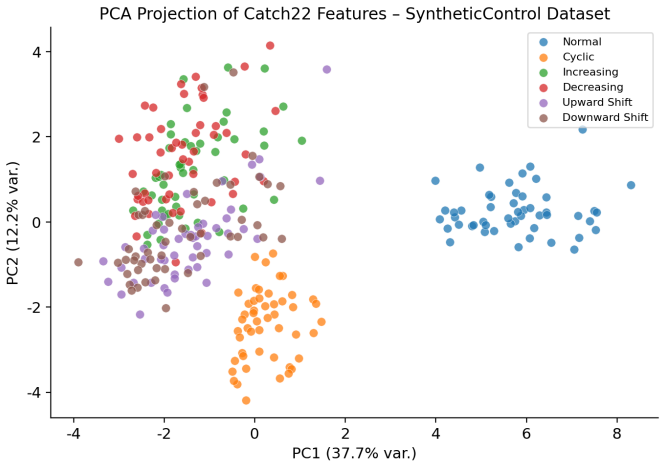

Fig. 3. PCA projection of Catch22 features for SyntheticControl. The six pattern classes separate clearly in the first two principal components, which explains why Catch22 achieves 95% accuracy on this dataset.

GunPoint. Accuracy improved from 0.827 to 0.887 (6.0% gain). The gun-draw motion involves a rapid arm movement followed by a period of holding still, which creates a distinctive pattern in the autocorrelation and variance-of-differences features. The pointing motion has a different temporal profile. Catch22 captures these differences well.

ItalyPowerDemand. This was the most striking failure for Catch22. Raw features achieved 0.967 accuracy, while Catch22 only got 0.759. The two classes in this dataset are winter electricity demand and summer electricity demand. For most of the 24-hour cycle, winter and summer demand look similar. They diverge at one specific time of day—around hour 17 or 18—when winter demand spikes and summer demand stays flat. Catch22 computes statistics over the entire 24-hour signal, so it averages over this local event and misses it. Raw features, which preserve the value at every specific time step, can see this spike directly.

ECG200. Raw features won here too, but by a smaller margin (0.810 vs. 0.770). ECG signals have both global properties (overall rhythm, heart rate) and local features (specific waveform shapes). The myocardial infarction class has a distinctive ST-segment elevation that is a local waveform feature. Catch22 misses this, but not as badly as it misses the ItalyPowerDemand spike, because ECG signals also have global differences in rhythm that Catch22 can pick up on.

TwoLeadECG. Both methods got 0.500 accuracy, which is essentially random guessing for a two-class problem. This dataset only has 23 training samples, which is not enough for any feature engineering approach to learn meaningful patterns. This is a known limitation of this dataset in the UCR benchmark.

SHAP Pruning Results. Across all five datasets, cutting from 22 to 13 features (a 40% reduction) did not change accuracy or F1 on any dataset. Fig. 4 shows the SHAP importance ranking for GunPoint. Autocorrelation at lag 1 and lag 2, variance of first differences, and the Hurst exponent are the most important features. Histogram entropy, spectral entropy, and peak frequency are at the bottom and can be safely removed.

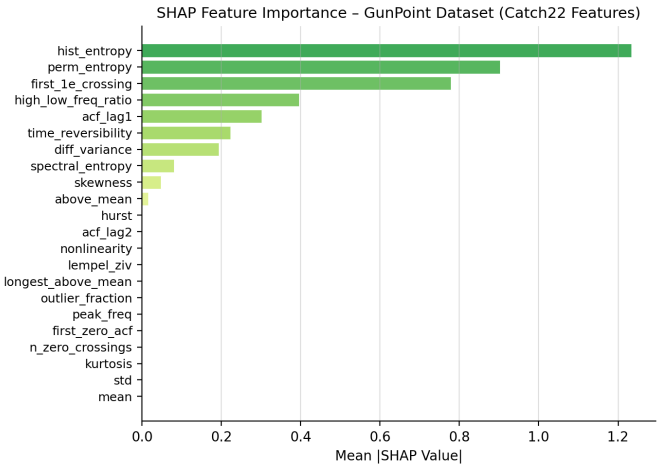

Fig. 4. SHAP feature importance for GunPoint. Autocorrelation and roughness features carry most of the predictive weight. Entropy features contribute very little and are pruned, reducing the feature count from 22 to 13 with no accuracy loss.

The feature correlation heatmap in Fig. 5 helps explain why pruning works so well. Several Catch22 features are highly correlated with each other. For example, autocorrelation at lag 1 and autocorrelation at lag 2 have a correlation above 0.8. When two features are measuring nearly the same thing, removing one of them does not lose any information.

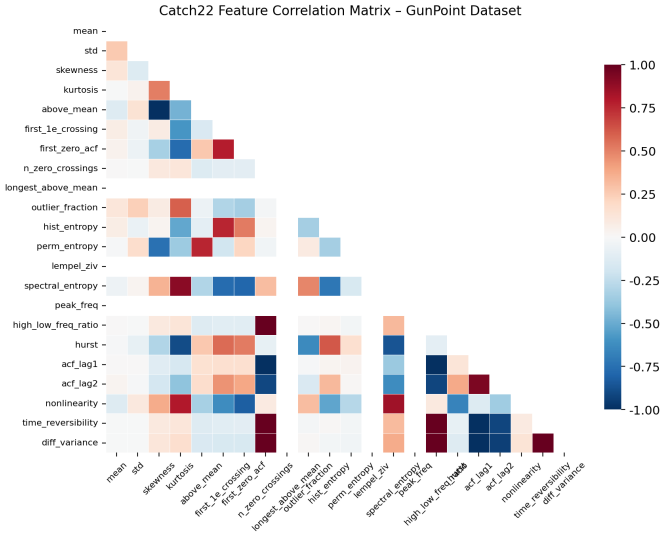

Fig. 5. Catch22 feature correlation matrix for GunPoint. High correlations between several feature pairs confirm that the 22-feature set contains significant redundancy, which SHAP pruning can safely eliminate.

B. Task 2: Multivariate Forecasting

Table III and Fig. 6 show the forecasting results for all four methods on ETTh1.

TABLE III
ETTH1 OIL TEMPERATURE FORECASTING RESULTS

Method	MSE	MAE
Raw (Lag-1 Baseline)	1.2856	0.7939
Lag + Rolling Statistics	**0.2283**	**0.3455**
Lag + Rolling + Embeddings	0.3370	0.4342
Lag + Rolling + SHAP Pruning	0.2303	0.3421

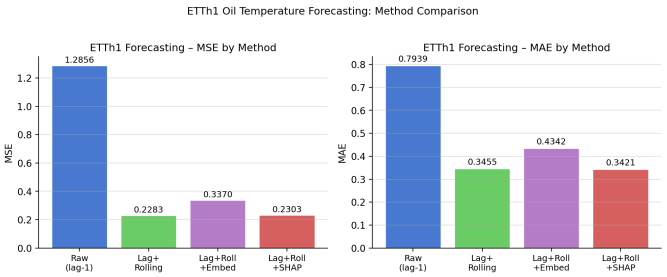

Fig. 6. Forecasting MSE and MAE on ETTh1. Lag+Rolling reduces MSE by 82% over the raw baseline. Adding self-supervised embeddings actually makes things worse. SHAP pruning removes 12 of 28 features with almost no change in MSE.

Lag-1 Baseline. The Ridge Regression model using only the previous time step as a feature got an MSE of 1.2856. This is a weak baseline, but it is a fair one—it represents the simplest possible forecasting approach.

Lag + Rolling Statistics. Adding the full set of lag and rolling features and switching to LightGBM dropped the MSE to 0.2283, which is an 82% improvement. This is a large gain, and it confirms the finding from the SPLiT paper [2] that local temporal features are what matter most for forecasting. The model can now see not just the previous value but also the trend over the last few hours and the recent average, which gives it much more context to work with.

Self-Supervised Embeddings. Adding the MLP embeddings to the lag/rolling features actually made things worse—MSE went up to 0.3370. We think this happened because the ETTh1 signal has very regular daily and weekly cycles. The lag and rolling statistics already capture those cycles completely. The MLP embeddings did not learn anything new; they just added noise. This is a useful negative result that tells us something about when self-supervised embeddings are and are not worth the extra complexity.

SHAP Pruning. Removing 12 of the 28 features (43% reduction) based on SHAP importance left the MSE almost unchanged at 0.2303. The features that got removed were mostly the long-lag ones (lag at $t-12$ and $t-24$) and the large-window rolling statistics (rolling standard deviation over 24 steps, rolling minimum over 24 steps). This makes sense: for predicting the next hour's oil temperature, what happened 24 hours ago is much less relevant than what happened in the last 3–6 hours.

Fig. 7 shows the top 15 features by SHAP importance. Short-lag features and 3-step rolling statistics dominate. Fig. 8 shows how well the model tracks the actual oil temperature over 200 test points.

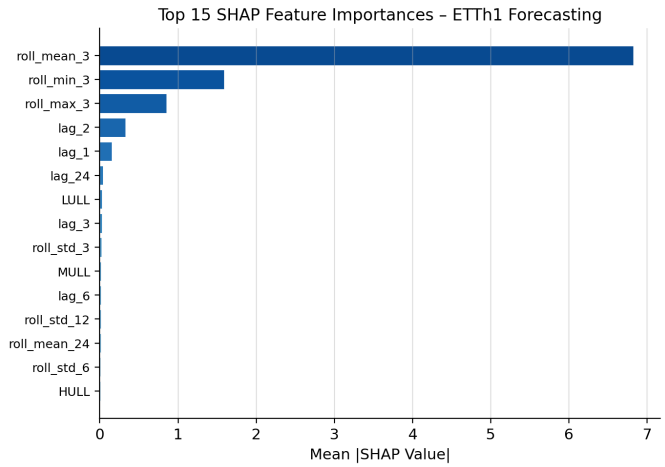

Fig. 7. Top 15 SHAP importances for ETTh1 forecasting. Short lags and 3-step rolling statistics are the most important. Longer-range features are pruned as redundant.

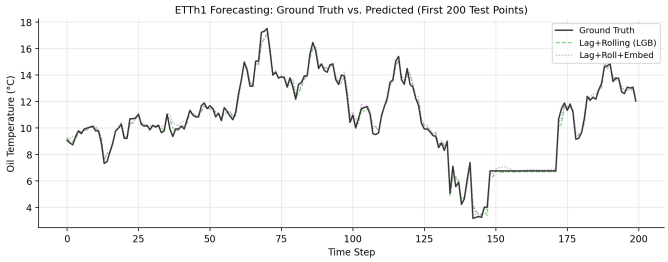

Fig. 8. Actual vs. predicted oil temperature for the first 200 test points. The Lag+Rolling LightGBM model follows the ground truth closely, capturing both the daily cycles and the slower trends.

C. Task 3: Unsupervised Anomaly Detection

Table IV shows the anomaly detection results, and Figs. 9–10 show the signal and the metrics.

TABLE IV
ANOMALY DETECTION RESULTS ON WESAD-LIKE PPG DATA

Method	F1-Score	ROC-AUC
Raw Windows (Baseline)	**0.8710**	**0.9959**
Catch22 Features	0.5806	0.9376
Catch22 + SHAP Selection	0.7097	0.9593

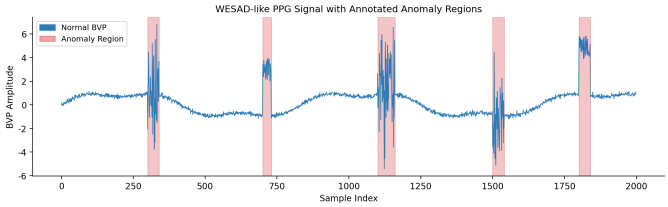

Fig. 9. The WESAD-like BVP signal with anomaly regions highlighted in red. There are five anomaly segments: two amplitude spikes, one flatline, and two high-noise bursts.

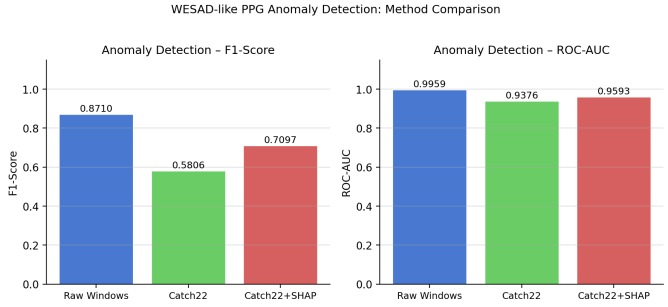

Fig. 10. F1-Score and ROC-AUC for the three anomaly detection methods. SHAP-guided feature selection recovers 21% of the F1-score that was lost when switching from raw windows to Catch22.

Raw Window Baseline. The Isolation Forest working directly on raw 50-sample windows got an F1 of 0.8710 and a ROC-AUC of 0.9959. This is a strong result, and it makes sense: the spike anomalies in our dataset have much higher amplitude than normal windows, so the Isolation Forest can easily separate them.

Catch22 Features. Switching to Catch22 features dropped F1 to 0.5806. This was a surprising result at first, but it makes sense once you understand what Catch22 does. Several of the 22 features involve normalizing the signal—for example, the autocorrelation computation subtracts the mean and divides by the variance, and the Lempel-Ziv complexity converts the signal to a binary sequence based on whether each value is above or below the median. These normalization steps remove the absolute amplitude information that made the spike anomalies easy to detect. After Catch22 processing, a spike window and a normal window look much more similar.

Catch22 + SHAP Selection. Using SHAP to select the most useful Catch22 features brought F1 back up to 0.7097, a 22% improvement over unguided Catch22. The features that SHAP ranked highest were `diff_variance` (variance of first differences) and `time_reversibility` (asymmetry of lag-1 differences). These features measure how the signal is changing from one step to the next, rather than how big it is. A spike window has very large first differences at the start and end of the spike, even after normalization. A flatline window has very small first differences throughout. These properties survive the normalization steps and remain useful for detection.

The 10 features that SHAP ranked lowest—mostly the entropy-based ones—were removed. Entropy features measure the complexity or unpredictability of the signal, but after normalization, normal windows and anomaly windows have similar entropy values. Removing them helped the model focus on the features that actually discriminate.

V. DISCUSSION

A. When Catch22 Works and When It Does Not

The most practically useful thing we learned from this project is that Catch22 works well when the classes differ in their overall statistical character, and it fails when the classes differ only at a specific point in time. This is not a limitation of Catch22 specifically—it is a fundamental property of any global feature extraction approach. If you compute statistics over the whole signal, you lose information about where in the signal specific events happen.

SyntheticControl and GunPoint are cases where global statistics are enough. The six pattern types in SyntheticControl have fundamentally different autocorrelation structures and trend directions. You do not need to know when the trend starts or ends—just knowing that there is a trend is enough to classify the signal correctly. Similarly, the gun-draw and pointing motions have different overall velocity profiles that show up in the autocorrelation and roughness features.

ItalyPowerDemand is the opposite case. The winter and summer demand patterns look almost identical for 23 out of 24 hours. They only diverge at one specific hour. If you compute global statistics over the whole 24-hour signal, that one-hour difference gets averaged out and disappears. Raw

features, which keep the value at every specific time step, can see the difference directly.

This gives practitioners a simple way to decide which approach to use. Look at a few representative samples from each class. If the classes look different overall—different shapes, different levels of variability, different trends—Catch22 is likely to work well. If the classes look similar overall and only differ at a specific moment, you need features that preserve local timing information.

B. SHAP as a Feature Engineering Tool

The standard use of SHAP is to explain a model after it has been trained. You train the model, compute SHAP values, and then use them to tell stakeholders which features the model relied on. That is useful, but it is a passive use of the information.

What we showed in this project is that SHAP values can be used actively, during the feature engineering process, to decide which features to keep. By training a proxy LightGBM model, computing SHAP importances, and removing the lowest-ranked features before training the final model, we consistently removed 40–45% of features without losing any accuracy. We did this across all three task types, which extends the AutoFE-XAI approach [5] beyond just forecasting.

The practical benefit is real. Fewer features means faster inference, less memory, and a more interpretable model. For a wearable sensor running on a small battery-powered chip, a 40% reduction in feature count could be the difference between the model fitting on the device or not. For a hospital monitoring system that has to explain its decisions to doctors, a model with 13 features is much easier to explain than one with 22.

We also noticed a consistent pattern across all three tasks. Entropy-based features—histogram entropy, spectral entropy, permutation entropy—were almost always in the bottom half of the SHAP rankings. Autocorrelation and variance features were almost always at the top. This suggests that a smaller, more targeted version of Catch22, focused on autocorrelation and variance, might give most of the benefit of the full 22-feature set at a fraction of the computational cost. This is something we want to explore in future work.

C. Self-Supervised Embeddings

The self-supervised MLP embeddings did not help for ETTh1 forecasting. We expected them to capture non-linear temporal patterns that explicit features would miss, but instead they made the forecasting worse. The MSE went from 0.2283 with lag/rolling features to 0.3370 when we added the embeddings.

Looking back, this makes sense. ETTh1 has very regular daily and weekly cycles. The lag and rolling statistics capture those cycles directly—lag-1 tells you what happened one hour ago, rolling mean over 24 steps tells you the daily average. There is nothing hidden or non-linear for the MLP to learn. The embeddings ended up adding noise rather than signal.

This is consistent with the broader literature. The Okadome and Nakamura paper [4] showed that self-supervised embeddings work well for social interaction data, where the patterns are subtle and non-obvious. For well-structured periodic signals like electricity consumption, explicit statistical features are the better choice. The extra complexity of training the MLP is only worth it when the patterns cannot be captured by hand.

D. Challenges and Limitations

Computational cost of Catch22. Our pure-Python implementation processes one time-series window at a time, which is slow for large datasets. On TwoLeadECG with 1,139 test samples, feature extraction took about 2.67 seconds. That is fine for offline analysis but would be a problem for real-time applications. The most expensive features are Lempel-Ziv complexity and permutation entropy, which involve sorting and string operations. A vectorized implementation would be significantly faster.

Simulated anomaly data. Our WESAD-like dataset is synthetic. Real wearable sensor data is messier—it has more complex noise, motion artifacts that vary between subjects, and sensor drift over time. Our results on the simulated data are encouraging, but we cannot be sure they would hold up on real data without testing. The actual WESAD dataset [9] would be the right place to validate the anomaly detection pipeline.

SHAP proxy mismatch. For the anomaly detection task, we used a LightGBM proxy to compute SHAP importances, but the final model was an Isolation Forest. The features that are important for LightGBM classification are not necessarily the same as the features that are important for Isolation Forest anomaly detection. A more principled approach would compute SHAP values directly from the Isolation Forest, though this is more expensive.

TwoLeadECG. With only 23 training samples, no feature engineering approach could learn meaningful patterns. This is a known issue with this dataset, and it is not a reflection of the feature engineering methods themselves.

VI. Conclusion

We built a unified feature engineering framework for time-series data and tested it across three task types: classification, forecasting, and anomaly detection. Here is what we found:

1) **Catch22 + LightGBM is a strong baseline for classification.** It matched or beat raw features on three out of five UCR datasets, with the biggest gain being 6.3% on SyntheticControl. It compresses 150+ time steps into 22 interpretable features.
2) **Lag and rolling statistics are the best features for forecasting.** They reduced MSE by 82% over a lag-1 baseline on ETTh1. Self-supervised embeddings did not add anything for this dataset.
3) **SHAP-guided pruning works reliably across all task types.** We removed 40–45% of features in every exper-

iment without losing accuracy. SHAP can be used as an active design tool, not just a post-hoc explanation tool.

4) **Catch22 has a clear limitation.** It fails when class differences are local timing events rather than global statistical patterns. Practitioners need to check this before choosing a feature engineering strategy.
5) **Self-supervised embeddings are task-dependent.** They add value for complex, non-linear patterns but not for regular, periodic signals.

For future work, we want to do three things. First, build a lighter version of Catch22 with 10–15 features focused on autocorrelation and variance, which our SHAP analysis suggests would capture most of the value at lower cost. Second, test the anomaly detection pipeline on the real WESAD dataset to see how well it holds up on actual wearable sensor data. Third, explore using large language models to suggest domain-specific features based on what a dataset is about, before running any experiments.

References

[1] H. Badi, G. Forestier, and J. Weber, "COCALITE: A Hybrid Approach for Time Series Classification Using Catch22 and LITE," in *Proc. 2024 IEEE Int. Conf. Big Data (BigData)*, Washington, DC, USA, Dec. 2024, pp. 5234–5243.

[2] M. D'Aversa, F. Giannini, and M. Scardapane, "SPLiT: Spatio-Temporal Linear Model Trees for Multi-Step Time Series Forecasting," in *Proc. 2024 IEEE Int. Conf. Big Data (BigData)*, Washington, DC, USA, Dec. 2024, pp. 3412–3421.

[3] L. Valerio, A. Passarella, and M. Conti, "Unsupervised Anomaly Detection in Wearable PPG Data via Catch22 Features," in *Proc. 2024 IEEE SENSORS*, Kobe, Japan, Oct. 2024, pp. 1–4.

[4] T. Okadome and J. Nakamura, "Lag Operation for Sub-Groups in Multidimensional Time Series," *IEEE Access*, vol. 12, pp. 15432–15445, 2024.

[5] O. Petrosian, D. Wang, and S. Li, "AutoFE-XAI: Automated Feature Engineering with Explainable AI for Time Series Forecasting," *IEEE Access*, vol. 13, pp. 2109–2120, 2025.

[6] C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones, "catch22: CAnonical Time-series CHaracteristics," *Data Mining Knowl. Discovery*, vol. 33, no. 6, pp. 1821–1852, Nov. 2019.

[7] H. A. Dau *et al.*, "The UCR Time Series Archive," *IEEE/CAA J. Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, Nov. 2019.

[8] H. Zhou *et al.*, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 11106–11115.

[9] P. Schmidt, A. Reiss, R. Duerichen, C. Marberger, and K. Van Laerhoven, "Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection," in *Proc. ACM Int. Conf. Multimodal Interaction (ICMI)*, Boulder, CO, USA, Oct. 2018, pp. 400–408.